

Chapter 7 – Segmentation by Clustering

Visual Computing

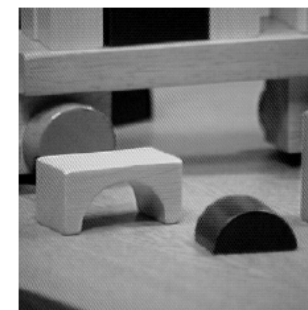
- Goal: Decompose an image (or a sequence of images) in a set of regions (1D, 2D or 3D), obtaining
 - Areas with a given semantic meaning, or
 - A set of boundary *pixels*, or
 - Structure of *pixels* with a given shape (lines, circles, polygons)
- Why to perform the segmentation?
 - To isolate regions of interest, for pos-processing.
 - Changing the image representation for a higher hierarchical level



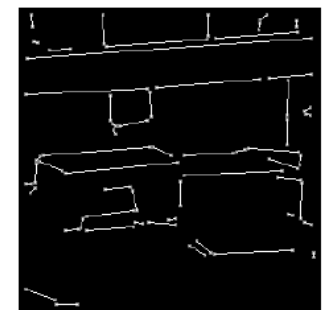
input



output



input

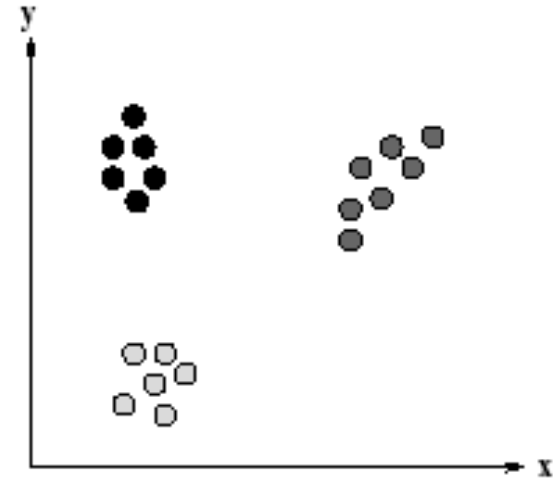


output

What is a region?

- Is a set of connected *pixels* with similar properties, regarding the *features* used for the operation of segmentation (gray level, color, texture, etc.);
- Adjacent regions should be significantly different, regarding the above *features*;
- The boundary of the regions should be smooth;
- The inner part of a region should be simple (low complexity), and should not contain a large number of holes;

- *Clustering*
 - Is the process of decomposition (classification) of a set of patterns (*feature vectors*) in different subsets, called *clusters*
- Examples of *features*
 - Intensity
 - Color
 - Texture
 - Shape
 - Motion
- Methods to solve the above problem
 - Classical Algorithms: K-means, EM, Isodata (Iterative Self-Organizing Data Analysis Techniques)
 - Based on Histograms: Otsu, Ohlander
 - Based on graph partition: Shi





Nº de clusters, $K = 5$

Goal:
$$D = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - m_k\|^2$$

Form K-means clusters from a set of n-dimensional vectors.

1. Set ic (iteration count) to 1.
2. Choose randomly a set of K means $m_1(1), m_2(1), \dots, m_K(1)$.
3. For each vector x_i compute $D(x_i, m_k(ic))$ for each $k = 1, \dots, K$ and assign x_i to the cluster C_j with the nearest mean.
4. Increment ic by 1 and update the means to get a new set $m_1(ic), m_2(ic), \dots, m_K(ic)$.
5. Repeat steps 3 and 4 until $C_k(ic) = C_k(ic + 1)$ for all k .



Form isodata clusters from a set of n -dimensional vectors.

1. assign x_i to the cluster l that minimizes

$$D_{\Sigma} = [x_i - m_l]' \Sigma_l^{-1} [x_i - m_l].$$

2. Merge clusters i and j if

$$|m_i - m_j| < \tau_v$$

where τ_v is a variance threshold.

3. Split cluster k if the maximum eigenvalue of σ_k is larger than τ_v .

4. Stop when

$$|m_i(t) - m_i(t+1)| < \epsilon$$

for every cluster i or when the maximum number of iterations has been reached.

Algorithm EM (Expectation Maximization)

Assumptions

We have a set of observations (e.g., trajectory samples)

$$X = (x_1, \dots, x_n), \quad \text{with} \quad x_i \in \mathbb{R}^D$$

$$\{x_n\}_{n=1}^N, \text{ independent and identical distributed (i.i.d.)} \quad \rightarrow \quad p(x_1, \dots, x_N) = \prod_{n=1}^N p(x_n | \theta)$$

$$p(x_n | \theta) = \sum_{m=1}^M w_{m,n} \mathcal{N}(x_n; \mu_m, P_m) = \sum_{m=1}^M w_{m,n} \frac{1}{(2\pi)^{D/2} |P_m|^{1/2}} \exp\left(-\frac{1}{2}(x_n - \mu_m)^\top P_m^{-1}(x_n - \mu_m)\right)$$

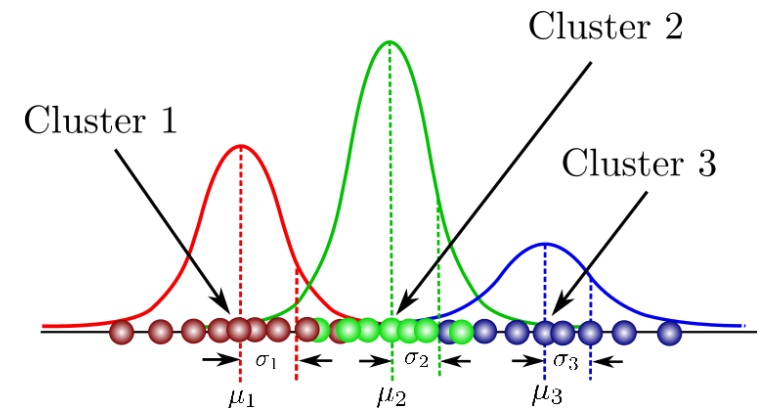
We also have

$$K = (k_1, \dots, k_M)$$

The goal is to estimate $p(X | \theta)$!

Why is difficult ?

$$p(X|\theta) = \sum_{K \in \{1, \dots, M\}^N} p(X, K | \theta)$$



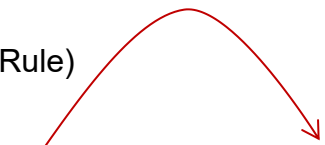
We have to first estimate of the complete log-likelihood. What is this ?

$$\ell(X, K, \theta) = \log p(X, K | \theta) = \log(p(X | K, \theta) p(K))$$

The aim is to compute the **expectation** (E-step) of the log-likelihood

$$\begin{aligned} \mathbb{E}_K \left\{ \ell(X, K, \theta) | X, \hat{\theta} \right\} &= \mathbb{E}_K \left\{ \log p(X, K | \theta) | X, \hat{\theta} \right\} \longrightarrow \boxed{\begin{aligned} Q(\theta, \hat{\theta}) &= \mathbb{E} \left[\log \ell(X, K | \theta) | X, \hat{\theta} \right] \\ &= \sum_K p(K | X, \theta) \log(X, K | \hat{\theta}) \end{aligned}} \\ &= \mathbb{E}_K \left\{ \log \prod_{n=1}^N p(x_n, K | \theta) | X, \hat{\theta} \right\} \\ &= \sum_{n=1}^N \mathbb{E}_K \left\{ \log p(x_n | K, \theta) + \log(p(K)) | X, \hat{\theta} \right\} \\ &= \sum_{n=1}^N \sum_{m=1}^M \left\{ w_{m,n} \log p(x_n | k_m, \theta) + w_{m,n} \log p(k_m) \right\} \\ &= \sum_{n=1}^N \sum_{m=1}^M w_{m,n} \left[-\frac{D}{2} \log 2\pi - \frac{1}{2} \log |P_m| - \frac{1}{2} (x_n - \mu_m)^\top P_m^{-1} (x_n - \mu_m) + \log C_m \right] \end{aligned}$$

How to compute the weights ? (Bayes Rule)

$$w_{m,n} = p(k_m | x_n, \hat{\theta}) = \frac{p(x_n | k_m, \hat{\theta}) p(k_m | \hat{\theta})}{p(x_n | \hat{\theta})} \propto \mathcal{N}(x_n; \mu_m, P_m)$$


Since we already have the weights, we now have to **maximize** (M-step) the objective function $Q(\cdot)$

$$Q(\theta, \hat{\theta}) = \sum_{n=1}^N \sum_{m=1}^M w_{m,n} [\log C_m + \log \mathcal{N}(x_n | \mu_m, P_m)]$$

$$\frac{\partial Q(\theta, \hat{\theta})}{\partial C_m} = 0 \quad \longrightarrow \quad \hat{C}_m = \frac{\sum_n w_{m,n}}{\sum_n \sum_m w_{m,n}} = \frac{\sum_n w_{m,n}}{n}$$

$$\frac{\partial Q(\theta, \hat{\theta})}{\partial \mu_m} = 0 \quad \longrightarrow \quad \hat{\mu}_m = \frac{\sum_n w_{m,n} x_n}{\sum_n w_{m,n}}$$

$$\frac{\partial Q(\theta, \hat{\theta})}{\partial P_m} = 0 \quad \longrightarrow \quad \hat{P}_m = \frac{\sum_n w_{m,n} (x_n - \hat{\mu}_m) (x_n - \hat{\mu}_m)^\top}{\sum_n w_{m,n}}$$

The EM Algorithm:

Step 1 (Initialization): Initialize the model parameters randomly, or we can use the results obtained with the K-means

Iterate between **E-step** and **M-step** as follows

Step 2 (Expectation step): compute the weights i.e.

$$w_{m,n} \propto \mathcal{N}(x_n; \mu_m, P_m)$$

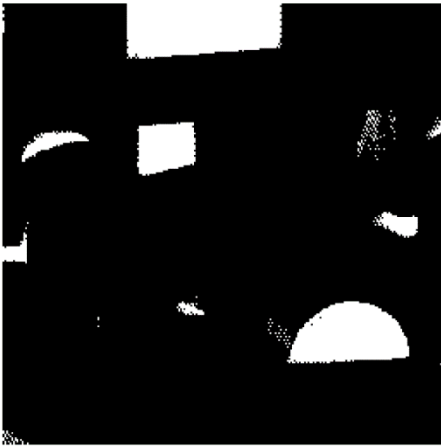
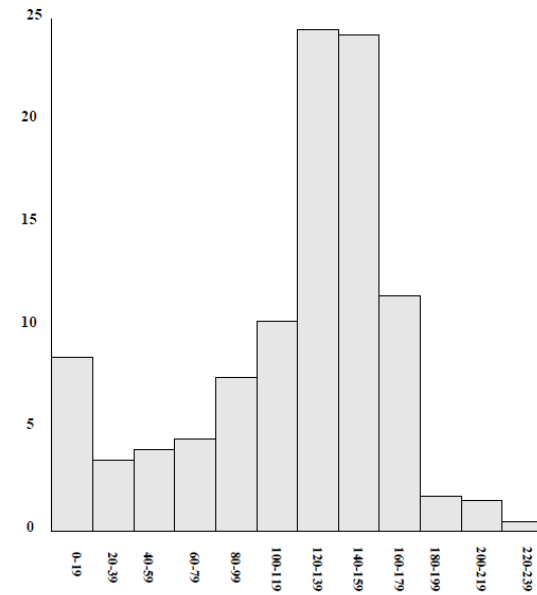
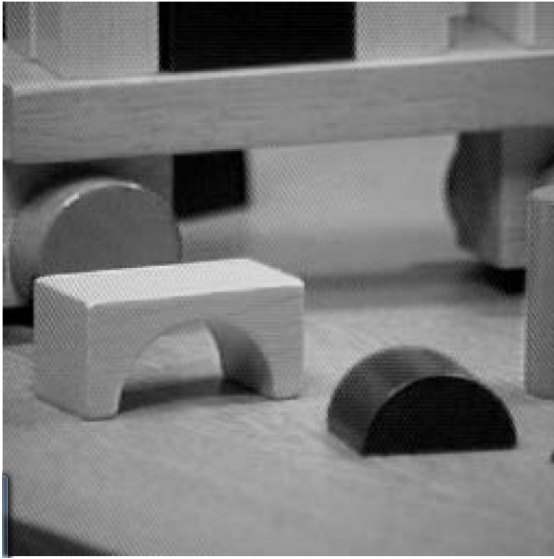
Step 3 (Maximization Step), compute the mean and the covariance of the mixture model, i.e.

$$\hat{C}_m = \frac{\sum_n w_{m,n}}{\sum_n \sum_m w_{m,n}} = \frac{\sum_n w_{m,n}}{n}$$

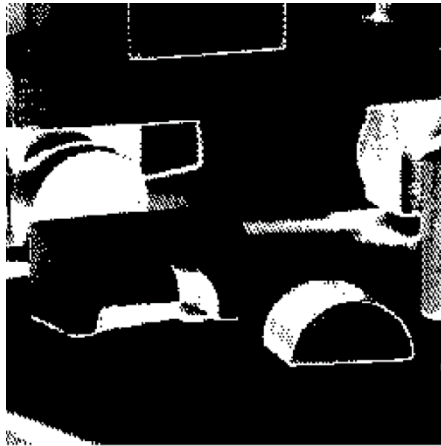
$$\hat{\mu}_m = \frac{\sum_n w_{m,n} x_n}{\sum_n w_{m,n}}$$

$$\hat{P}_m = \frac{\sum_n w_{m,n} (x_n - \hat{\mu}_m) (x_n - \hat{\mu}_m)^\top}{\sum_n w_{m,n}}$$

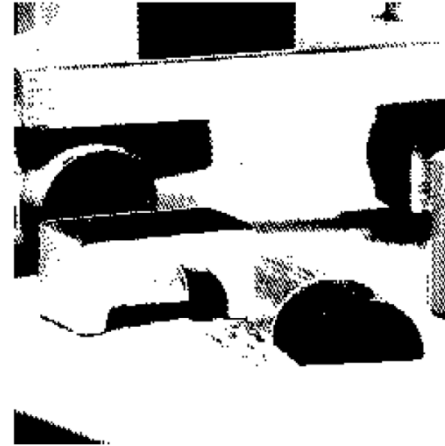
Methods based on Histograms



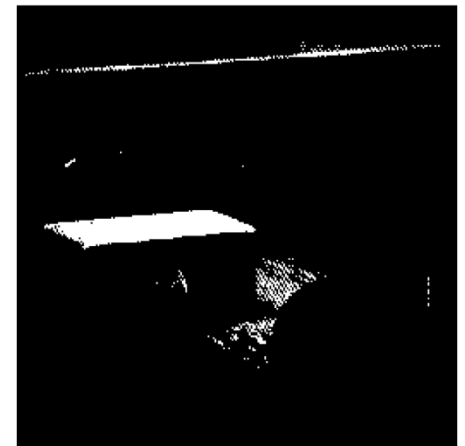
$0 < Ths < 30$



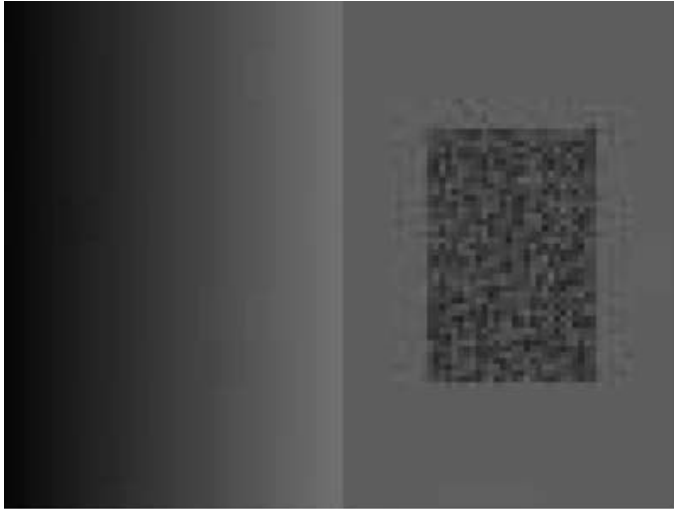
$31 < Ths < 100$



$101 < Ths < 179$

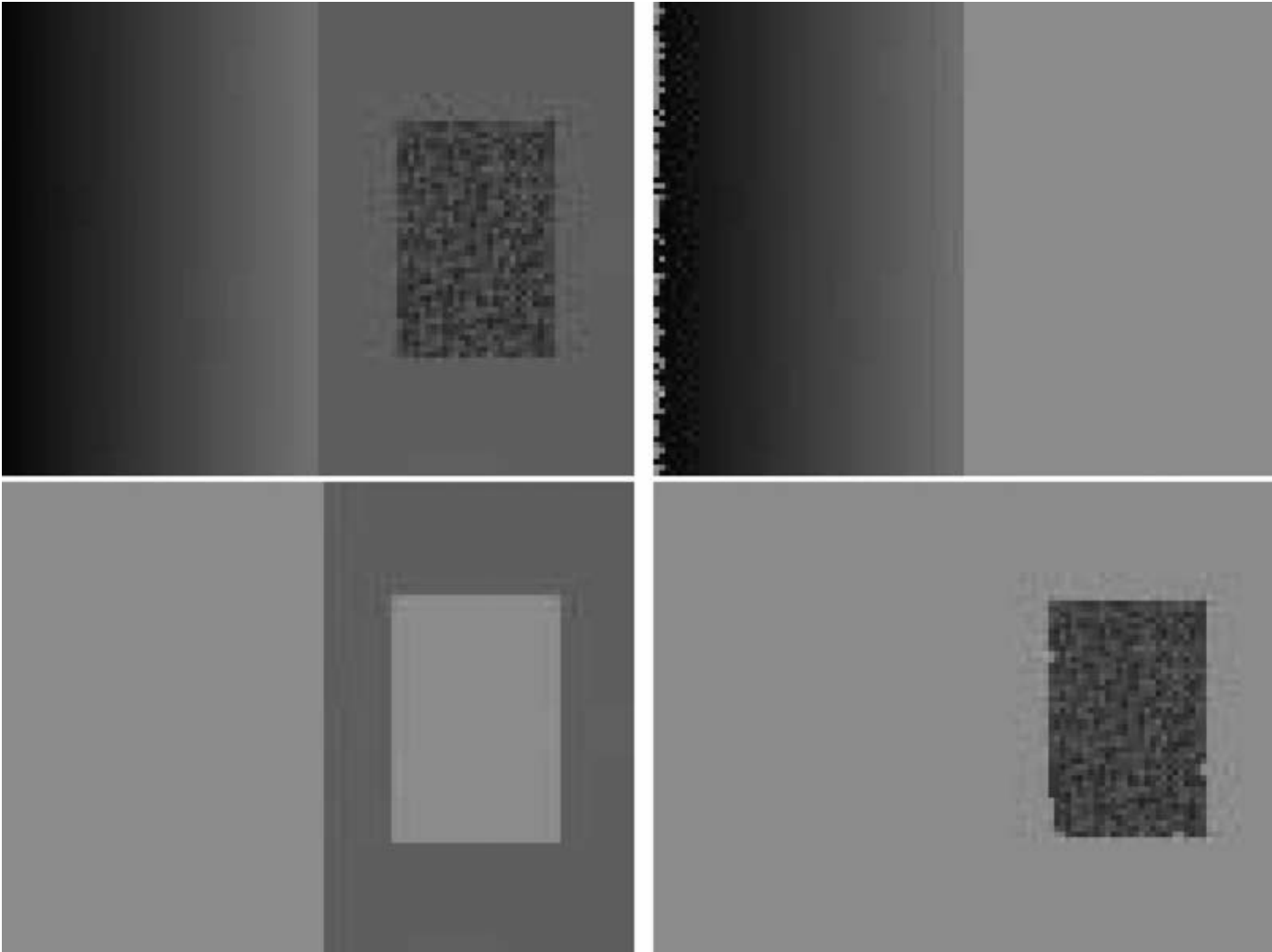


$180 < Ths < 239$



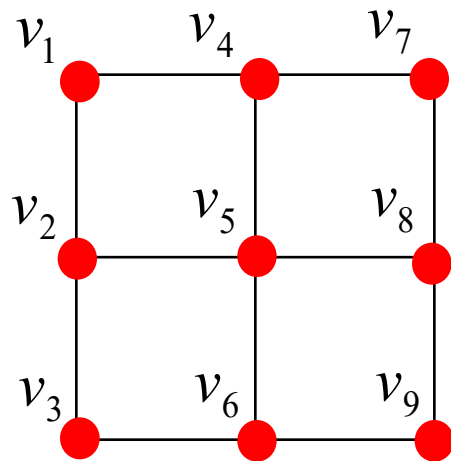
Goal: We want to know which are the regions in the image

Region segmentation using Graphs



Three regions !!

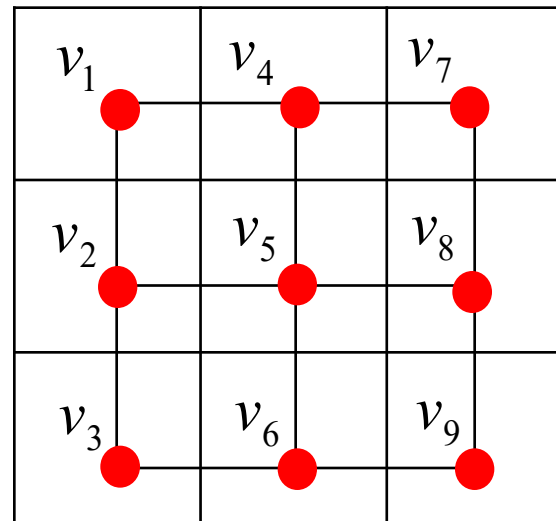
Region segmentation using Graphs



Vertices - $V = (v_1, \dots, v_9)$

Edges - $E = \{(v_1, v_2), (v_2, v_3), (v_1, v_4), (v_2, v_5), (v_3, v_6), (v_4, v_5), (v_5, v_6), (v_4, v_7), (v_5, v_8), (v_6, v_9), (v_7, v_8), (v_8, v_9)\}$

Weights - w_{v_i, v_j}

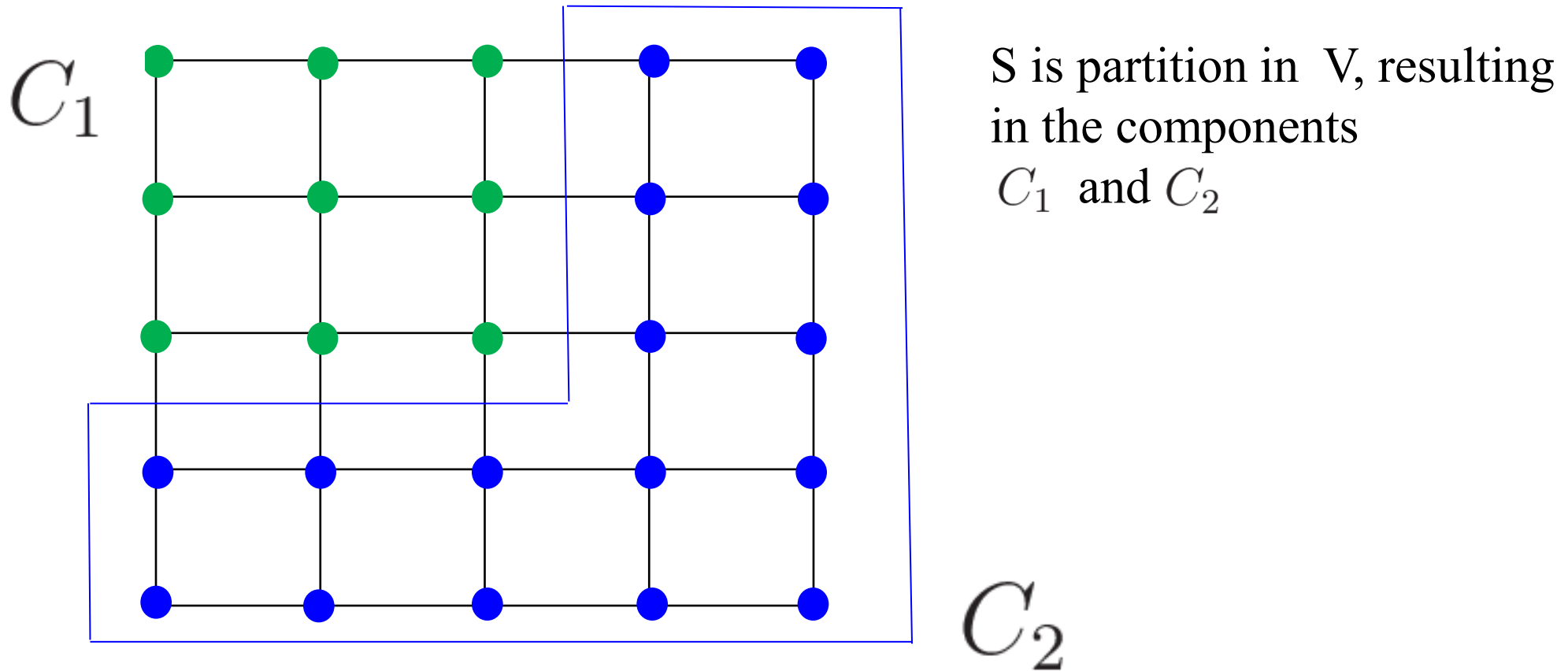


$$w(v_i, v_j) = |I(p_i) - I(p_j)|$$

In the image context, we have

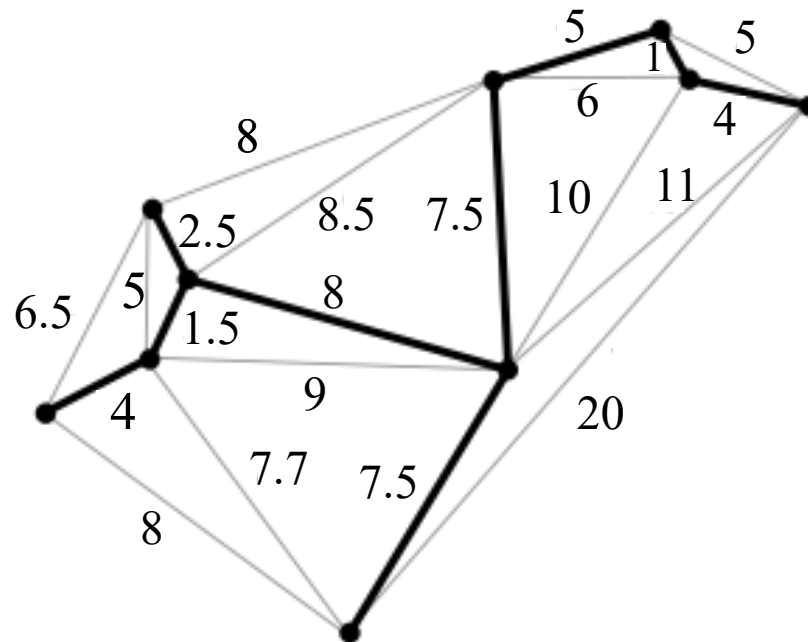
Representing the segmentation

The goal is to obtain a segmentation S , that can be illustrated as:



Minimum Spanning Tree

The **minimum spanning tree (MST)** is the tree that is generated in the graph which links the vertices with the minimal cost in its edges (with no cycles within).



Hint: for each node, select the neighbor node with smaller weight in the edge!

The algorithm uses two definitions

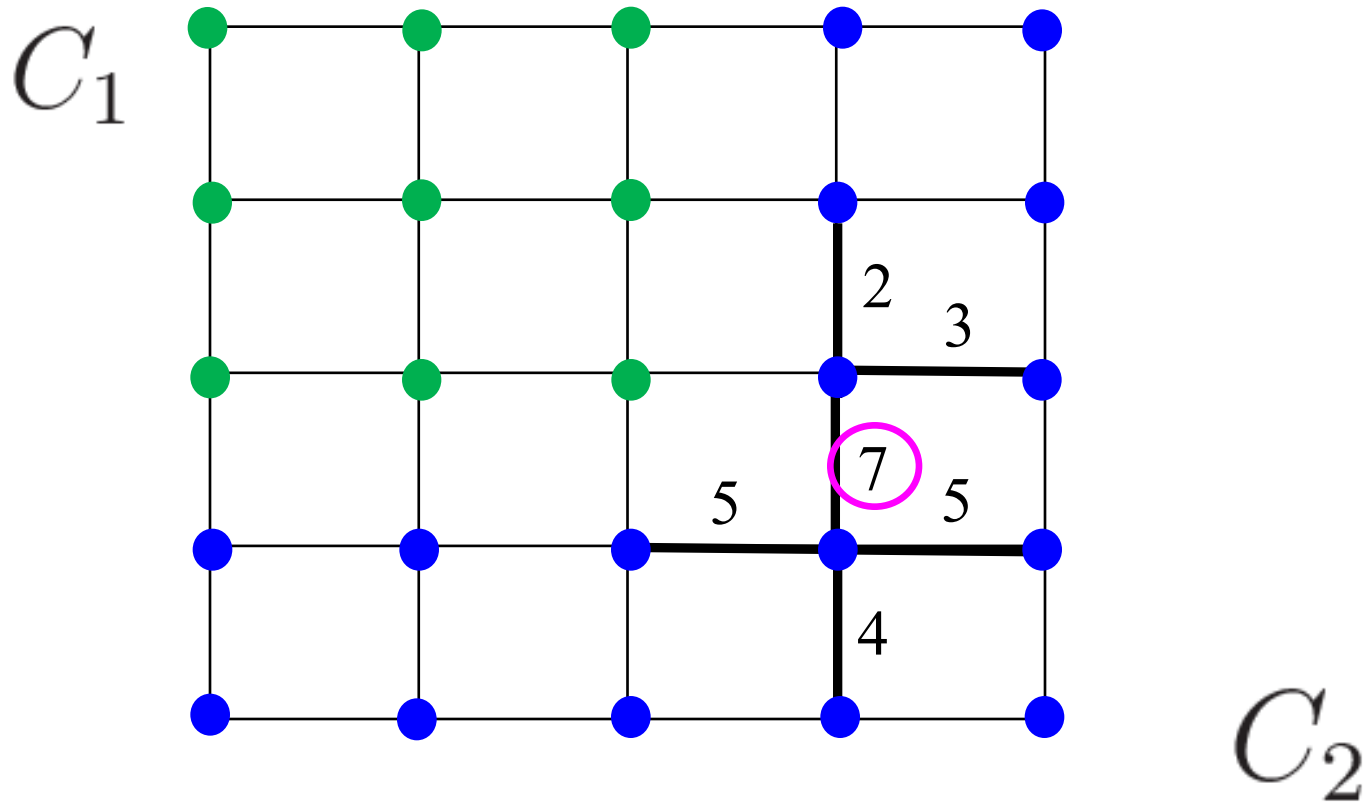
1. Internal difference of a component – $Int(C)$

$$Int(C) = \max_{e \in MST(C,E)} w(e)$$

2. Difference between two components – $Dif(C_1, C_2)$

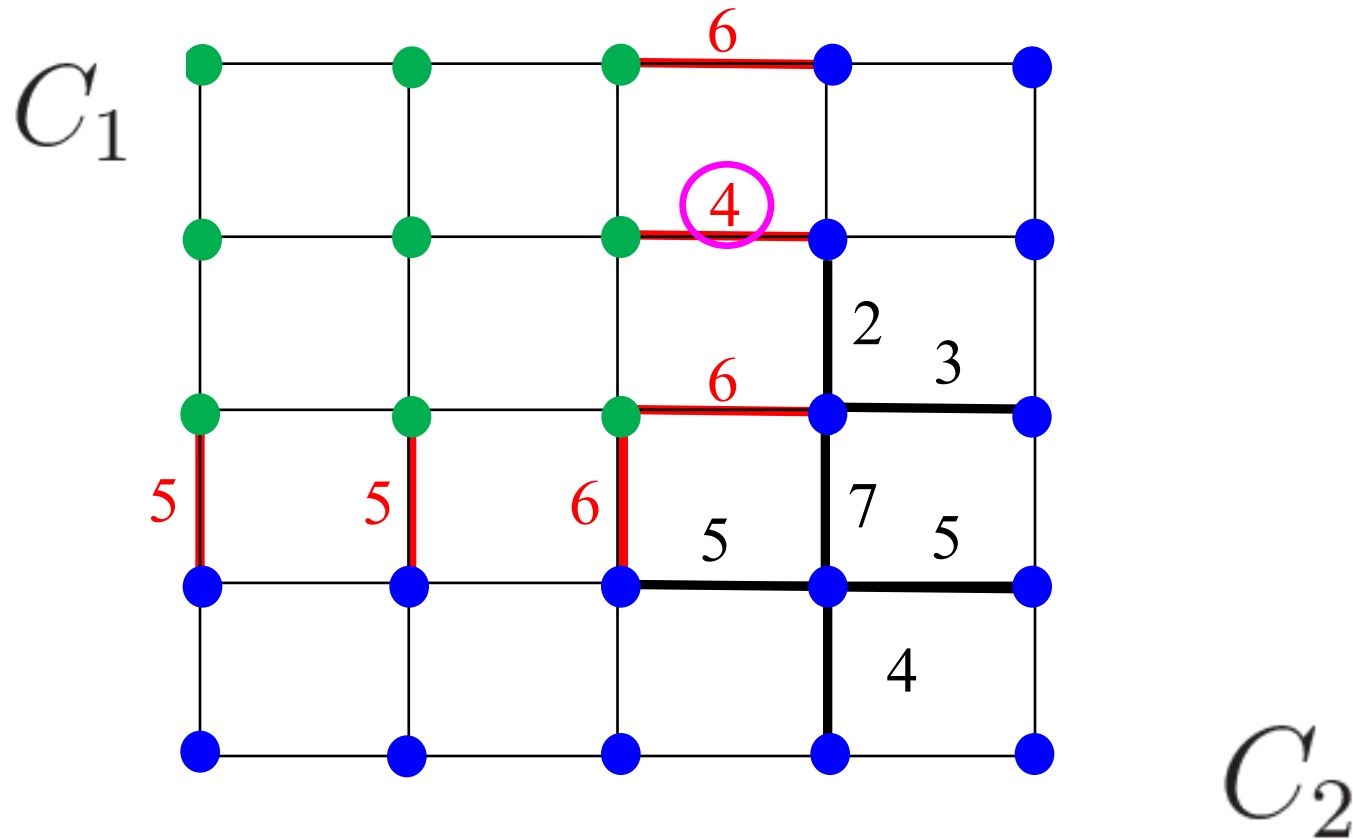
$$Dif(C_1, C_2) = \min_{\substack{v_i \in C_1, \\ v_j \in C_2, \\ (v_i, v_j) \in E}} w(v_i, v_j)$$

$$Int(C) = \max_{e \in \text{MST}(C,E)} w(e)$$



Example

$$Dif(C_1, C_2) = \min_{\substack{v_i \in C_1, \\ v_j \in C_2, \\ (v_i, v_j) \in E}} w(v_i, v_j)$$



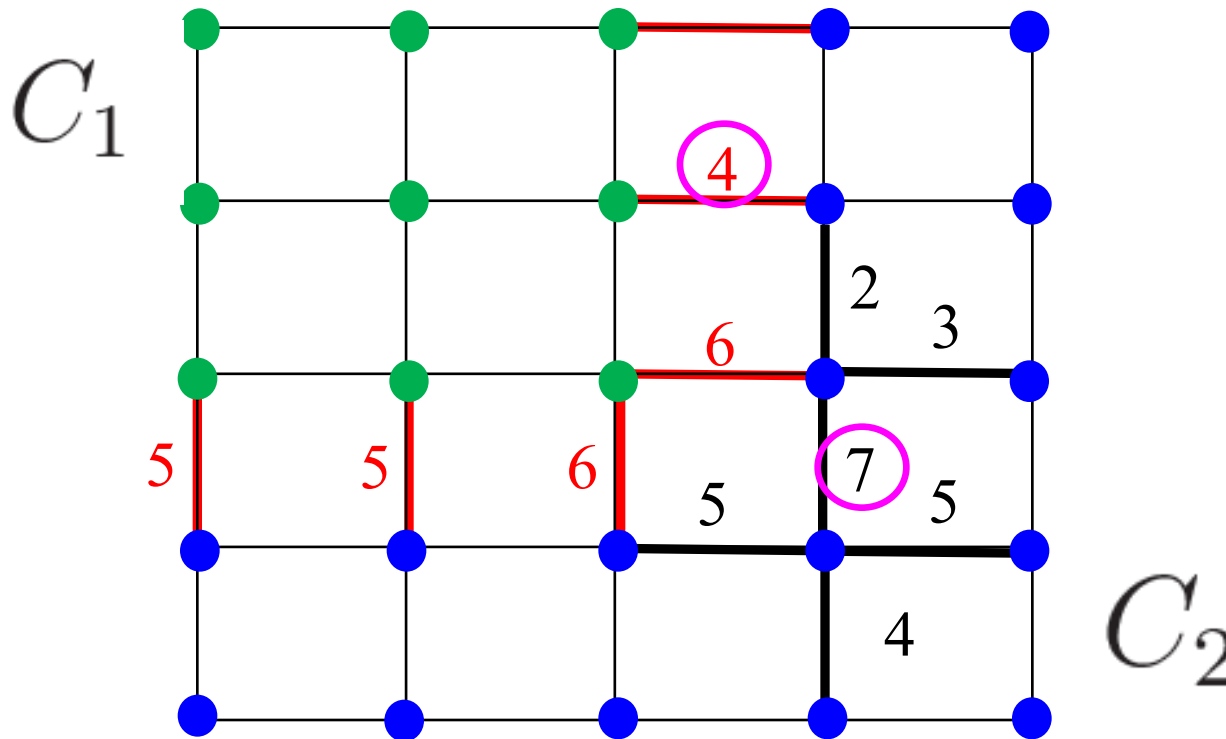
Merge criterion of two regions

There is a boundary between two regions if:

$$D(C_1, C_2) = \begin{cases} \text{true} & \text{if } Dif(C_1, C_2) > MInt(C_1, C_2) \\ \text{false} & \text{caso contrário} \end{cases}$$

where we define

$$MInt(C_1, C_2) = \min[Int(C_1) + \tau(C_1), Int(C_2) + \tau(C_2)]$$



Algorithm 1 Efficient Graph-based Image Segmentation

Sort E into $\pi = (o_1, \dots, o_m)$, by non-decreasing edge weight

Start with a new segmentation S^0 , where each vertex v_i is in its component

for $q = 1 : m$ **do**

 Construct S^q given S^{q-1} as follows:

- Let v_i and v_j denote vertices connected by the q th edge, i.e., $o_q = (v_i, v_j)$
- Let C_i^{q-1} containing v_i and C_j^{q-1} containing v_j

if $C_i^{q-1} \neq C_j^{q-1}$ **AND** $w(o_q) \leq MInt(C_i^{q-1}, C_j^{q-1})$ **then**

 Update S^q by MERGING (C_i^{q-1}, C_j^{q-1})

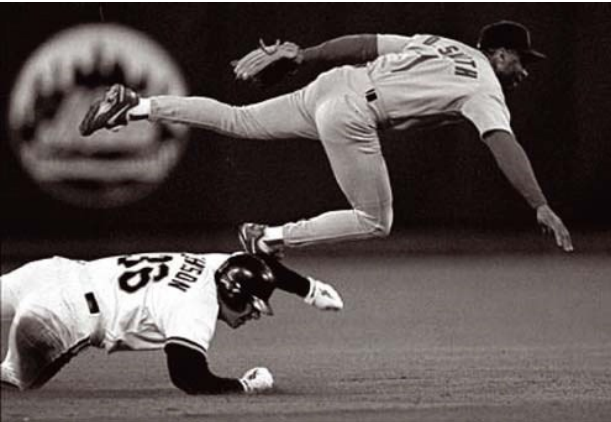
else

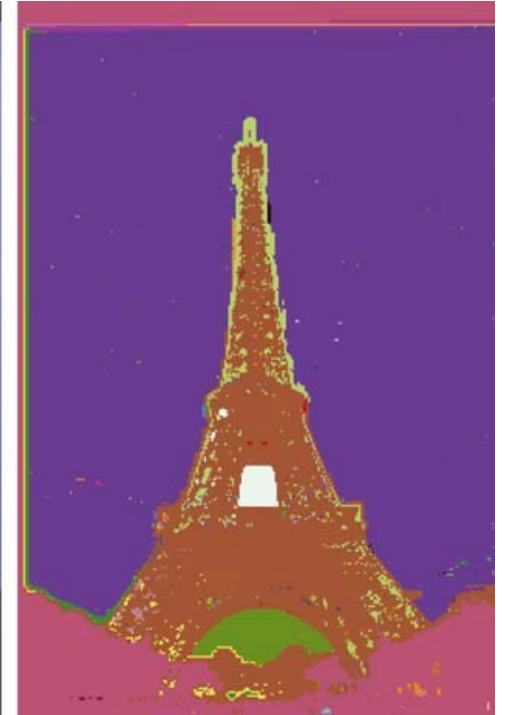
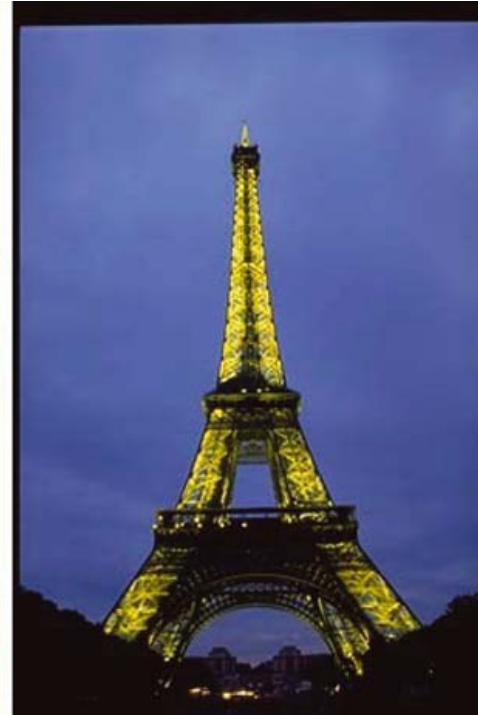
$S^q = S^{q-1}$

end if

end for

Return $S = S^m$





- The main idea is:
 - Start a new region with a single pixel (e.g., top-left point in the image) and then, start to perform *pixels* clustering in that region while the *similarity measures are significant large*.
 - Initialize a new region when the above criterion fails

- Let R be the region with N pixels with intensity y
- The mean and covariance of a region are computed as follows:

$$\bar{X} = \frac{1}{N} \sum_{(r,c) \in R} I(r,c)$$

$$S^2 = \sum_{(r,c) \in R} (I(r,c) - \bar{X})^2$$

- Assuming statistical independence between the pixels in R and the intensity y , the statistical criterion is given by

$$T = \left[\frac{(N-1)N}{N+1} (y - \bar{X})^2 / S^2 \right]^{\frac{1}{2}} \quad \begin{array}{ll} T < \tau & \text{similar} \\ T \geq \tau & \text{non-similar} \end{array}$$

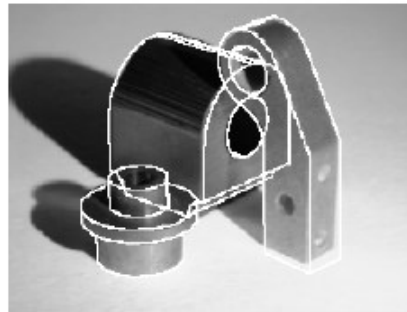
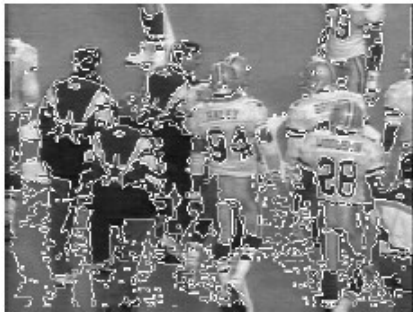
- If the T criterion is small, the intensity “y” (i.e., the pixel intensity) is added to the region R
- Finally, the mean and covariance are updated as follows:

$$\bar{X}_{new} \leftarrow (N\bar{X}_{old} + y)/(N + 1)$$

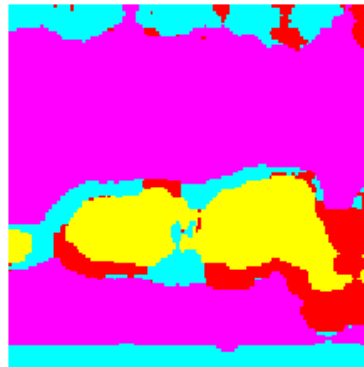
$$S_{new}^2 \leftarrow S_{old}^2 + (y - \bar{X}_{new})^2 + N(\bar{X}_{new} - \bar{X}_{old})^2$$

Region representation

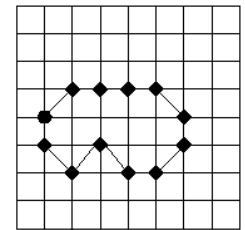
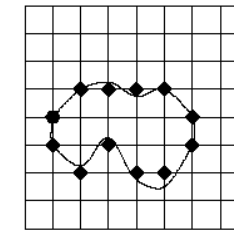
- *Overlays*



- Image with *labels*

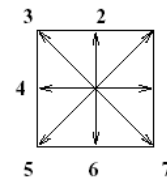


- *Chain codes*



original curve

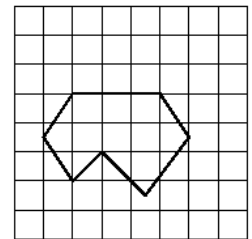
chain code links



encoding scheme

100076543532

chain code representation



polygonal approximation

- *Quadtrees*

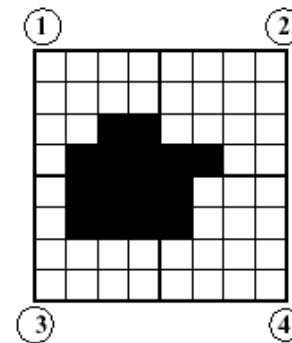
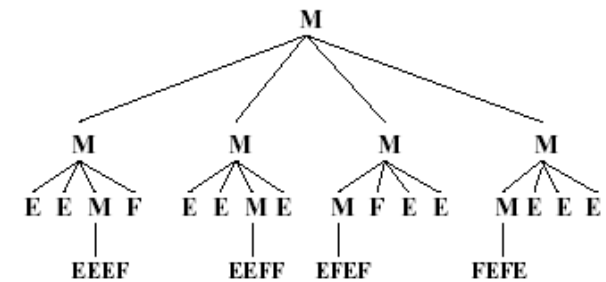


image region



quad tree representation

Boundary following (border tracking)

```

procedure border(S);
{
  for R:= 1 to NLines
  {
    for C:= 1 to NPIXELS
    {
      LABEL:= S[R,C];
      if new_region(LABEL) then add(CURRENT,LABEL);
      NEIGHB:= neighbors(R,C,LABEL);
      T:= pixeltype(R,C,NEIGHB);
      if T == 'border'
      then for each pixel N in NEIGHB
      {
        CHAINSET:= chainlist(LABEL);
        NEWCHAIN:= true;
        for each chain X in CHAINSET while NEWCHAIN
          if N==rear(X)
          then { add(X,[R,C]); NEWCHAIN:= false }
        if NEWCHAIN
        then make_new_chain(CHAINSET,[R,C],LABEL);
      }
    }
  }
  for each region REG in CURRENT
  if complete(REG)
  then { connect_chains(REG); output(REG); free(REG) }
}

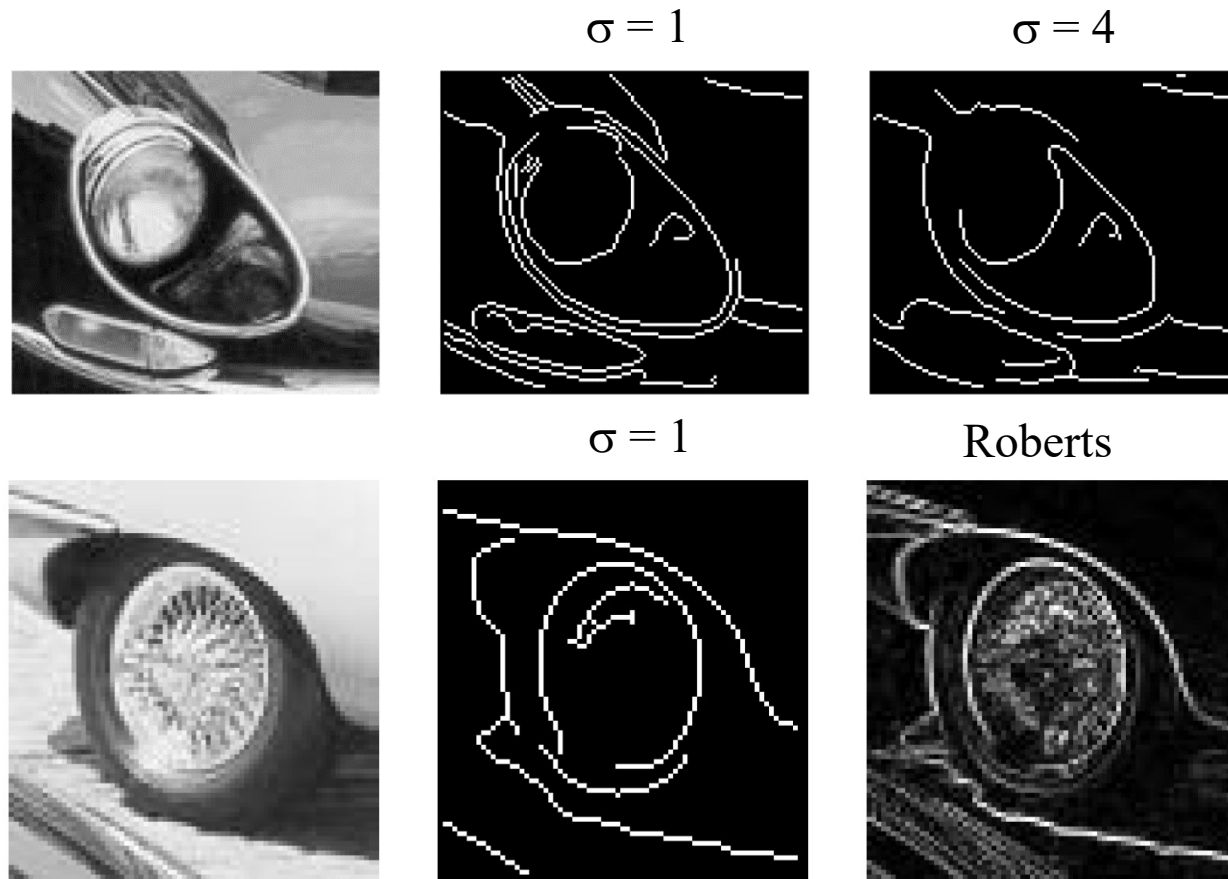
```

	1	2	3	4	5	6	7
1	0	0	0	0	0	0	0
2	0	0	0	0	2	2	0
3	0	1	1	1	2	2	0
4	0	1	1	1	2	2	0
5	0	1	1	1	2	2	0
6	0	0	0	0	2	2	0
7	0	0	0	0	0	0	0

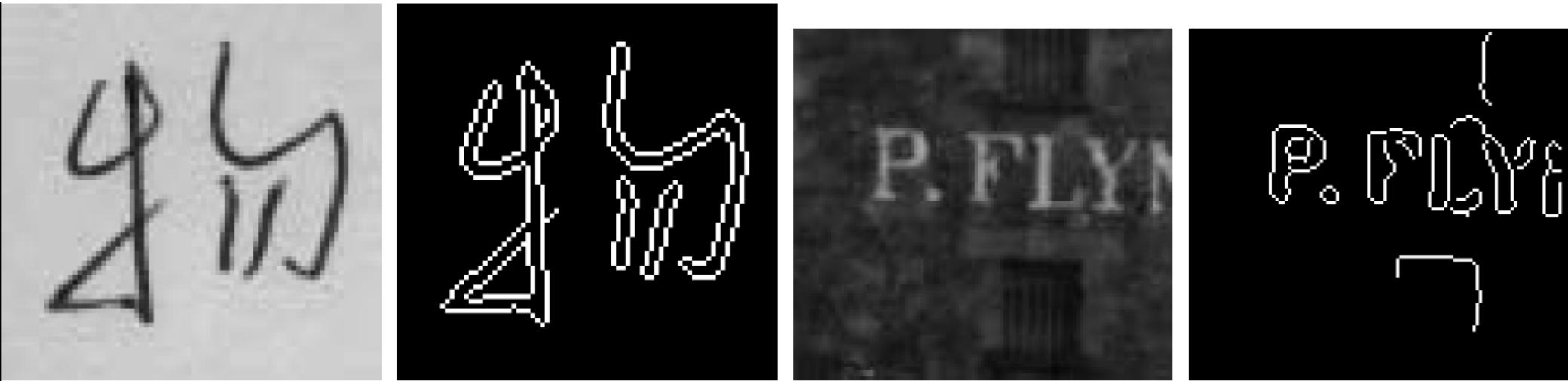
Region	Length	List
1	8	(3,2)(3,3)(3,4)(4,4)(5,4)(5,3)(5,2)(4,2)
2	10	(2,5)(2,6)(3,6)(4,6)(5,6)(6,6)(6,5)(5,5) (4,5)(3,5)

Boundary detection – Canny Algorithm

- The “Canny edge detector” extracts the boundary segments in an image of intensities
- It provides contour segments and can be tuned with a smooth operation (σ)

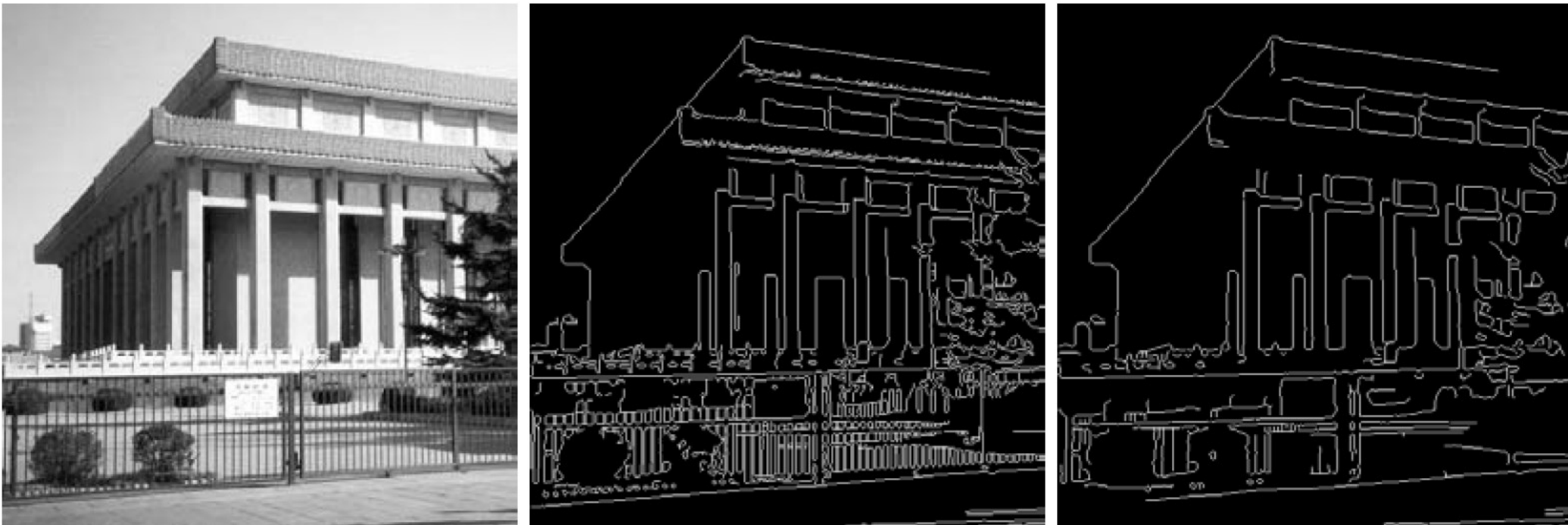


Boundary detection – Canny Algorithm



The region identification associated with symbols or surfaces are easy to obtain given a good image contrast!

Boundary detection – Canny Algorithm



Mao's Tomb Building in Beijing. Obtained contours with Canny algorithm ($\sigma = 1$ e $\sigma = 2$). Notice that, some of the 3D structures are well defined and some boundaries come from shadows and lighter regions

Compute thin connected edge segments in the input image.

$\mathbf{I}[\mathbf{x}, \mathbf{y}]$: input intensity image; σ : spread used in Gaussian smoothing;
 $\mathbf{E}[\mathbf{x}, \mathbf{y}]$: output binary image;
 $\mathbf{IS}[\mathbf{x}, \mathbf{y}]$: smoothed intensity image;
 $\mathbf{Mag}[\mathbf{x}, \mathbf{y}]$: gradient magnitude; $\mathbf{Dir}[\mathbf{x}, \mathbf{y}]$: gradient direction;
 T_{low} is low intensity threshold; T_{high} is high intensity threshold;

- Main ideas:
 - Non-maximum suppression
 - Tracking with hysteresis

procedure Canny($\mathbf{I}[]$, σ);

- 1) **procedure** Suppress_Nonmaxima($\mathbf{Mag}[]$, $\mathbf{Dir}[]$);
- 2) **procedure** Edge_Detect($\mathbf{Mag}[]$, T_{low} , T_{high} , $\mathbf{E}[]$);
- 3) **procedure** Follow_Edge(x, y , $\mathbf{Mag}[]$, T_{low} , T_{high} , $\mathbf{E}[]$);

Canny Algorithm

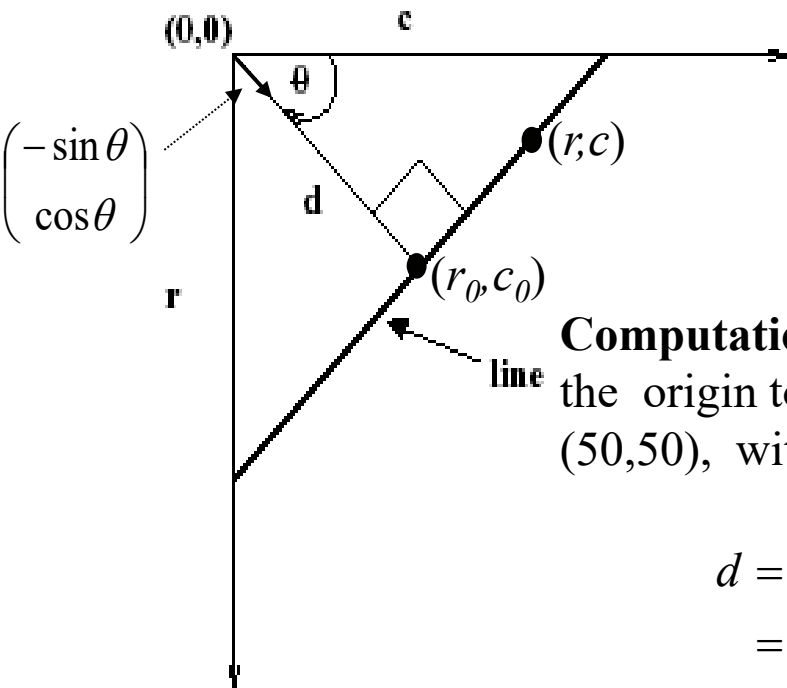
```
procedure Canny( I[],  $\sigma$ );  
{  
  IS[] = image I[] smoothed by convolution with Gaussian  $G_\sigma(x, y)$ ;  
  use Roberts operator to compute Mag[ $x, y$ ] and Dir[ $x, y$ ] from IS[];  
  Suppress_Nonmaxima( Mag[], Dir[],  $T_{low}$ ,  $T_{high}$  );  
  Edge_Detect( Mag[],  $T_{low}$ ,  $T_{high}$ , E[] );  
}
```

```
procedure Suppress_Nonmaxima( Mag[], Dir[] );  
{  
  define +Del[4] = (1,0), (1,1), (0,1) (-1,1);  
  define -Del[4] = (-1,0), (-1,1), (0,-1) (1,-1);  
  for x := 0 to MaxX-1;  
  for y := 0 to MaxY-1;  
  {  
    direction := ( Dir[ $x, y$ ] +  $\pi/8$  ) modulo  $\pi/4$ ;  
    if ( Mag[ $x, y$ ]  $\leq$  Mag[ $x, y$ ] + Del[direction]) then Mag[ $x, y$ ] := 0;  
    if ( Mag[ $x, y$ ]  $\leq$  Mag[ $x, y$ ] + -Del[direction]) then Mag[ $x, y$ ] := 0;  
  }  
}
```

```
procedure Edge_Detect( Mag[],  $T_{low}$ ,  $T_{high}$ , E[] );  
{  
  for x := 0 to MaxX - 1;  
  for y := 0 to MaxY - 1;  
  {  
    if (Mag[ $x, y$ ]  $\geq T_{high}$ ) then Follow_Edge(  $x, y$ , Mag[],  $T_{low}$ ,  $T_{high}$ , E[] );  
  } ;  
}
```

```
procedure Follow_Edge(  $x, y$ , Mag[],  $T_{low}$ ,  $T_{high}$ , E[] );  
{  
  E[ $x, y$ ] := 1;  
  while Mag[ $u, v$ ]  $> T_{low}$  for some 8-neighbor [ $u, v$ ] of [ $x, y$ ]  
  {  
    E[ $u, v$ ] := 1;  
    [ $x, y$ ] := [ $u, v$ ];  
  } ;  
}
```

Hough Transform – Extraction of line segments



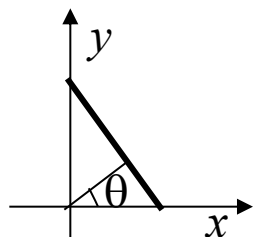
Line equation:

$$d = -r \sin \theta + c \cos \theta$$

Computation of “d”: if the perpendicular of the origin to the line passes through the point (50,50), with an angle of 315° then

$$\begin{aligned} d &= -50 \sin(315^\circ) + 50 \cos(315^\circ) \\ &= -50(-.707) + 50(.707) \\ &\approx 70 \end{aligned}$$

Coordinate systems xy:



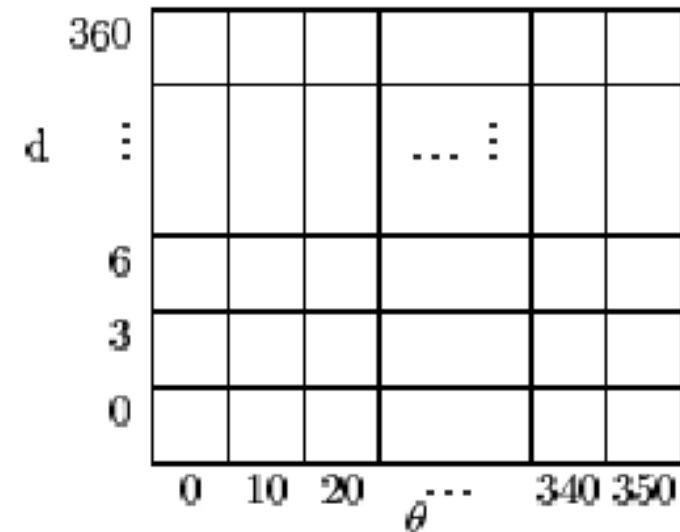
Line equation:

$$d = x \cos \theta + y \sin \theta$$

$$x \rightarrow c$$

$$y \rightarrow -r$$

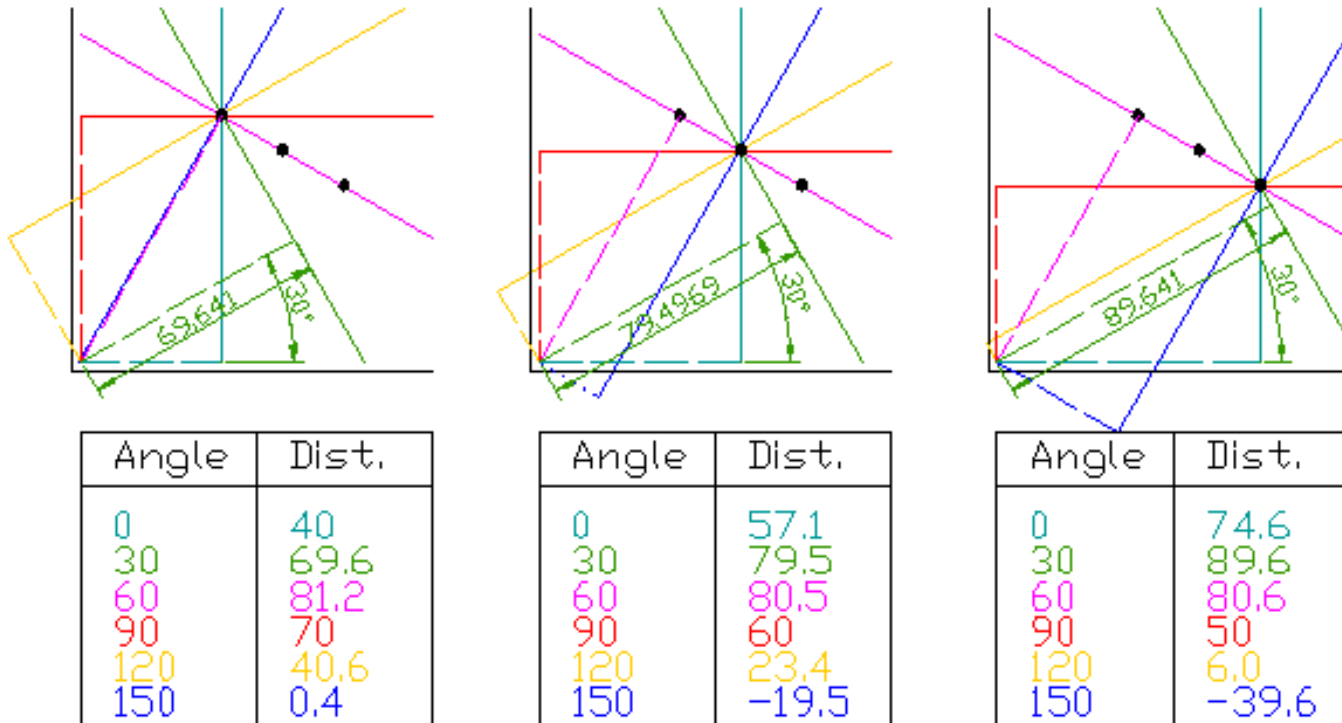
Accumulator $A(d_q, \theta_q)$



H: 256x256 image

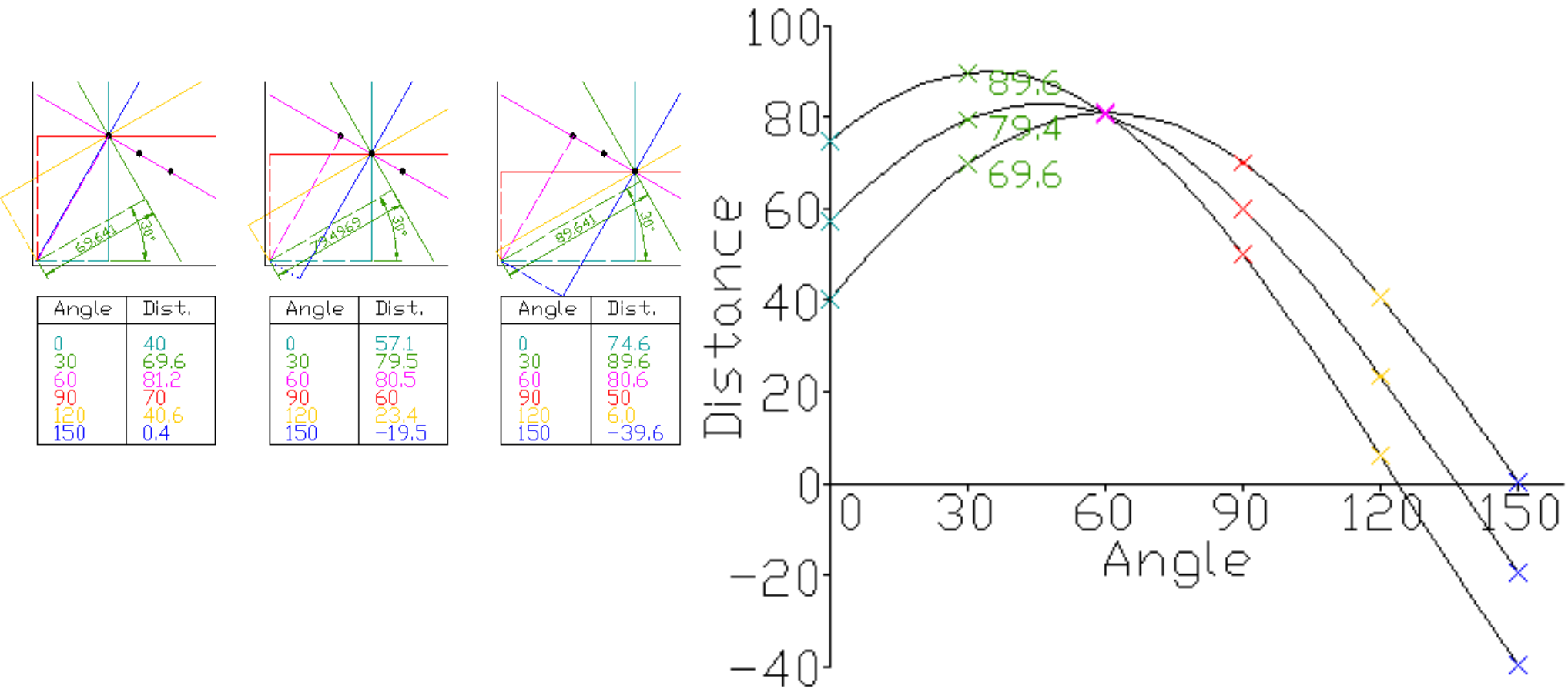
Example

Let us consider 3 points (in black) – Goal: Determine the line that links them !



1. For each point, define a n° of lines (six) that passes through that point in different pre-defined angles
2. For each line, there exists its correspondent perpendicular (dashed), that intersects the origin
3. For each line, we compute the length (perpendicular distance to the origin – dashed)
4. Repeat the above procedure for each point

Example



The point that the curves intersect, give us the **distance** and the **angle**! In this case the lines intersect the pink point; that corresponds to the **pink line**!

Hough transform - Algorithm

Accumulate the straight line segments in gray-tone image S to accumulator A.

S[R,C] is the input gray-tone image.

NLINES is the number of rows in the image.

NPIXELS is the number of pixels per row.

A[DQ,THETAQ] is the accumulator array.

DQ is the quantized distance from a line to the origin.

THETAQ is the quantized angle of the normal to the line.

```
procedure accumulate_lines(S,A);
{
  A := 0;
  PTLIST := NIL;
  for R := 1 to NLINES
    for C := 1 to NPIXELS
      {
        DR := row_gradient(S,R,C);
        DC := col_gradient(S,R,C);
        GMAG := gradient(DR,DC);
        if GMAG > gradient_threshold
          {
            THETA := atan2(DR,DC);
            THETAQ := quantize_angle(THETA);
            D := abs(C*cos(THETAQ) - R*sin(THETAQ));
            DQ := quantize_distance(D);
            A[DQ,THETAQ] := A[DQ,THETAQ]+GMAG;
            PTLIST(DQ,THETAQ) := append(PTLIST(DQ,THETAQ),[R,C])
          }
      }
}
```

Algorithm 26: Hough Transform for Finding Straight Lines

Example

	1	2	3	4	5
1	0	0	0	100	100
2	0	0	0	100	100
3	0	0	0	100	100
4	100	100	100	100	100
5	100	100	100	100	100

gray level image

	1	2	3	4	5
1	0	0	300	300	0
2	0	0	300	300	0
3	0	0	200	200	0
4	0	0	100	100	0
5	0	0	0	0	0

column gradient

	1	2	3	4	5
1	0	0	0	0	0
2	0	0	0	0	0
3	300	300	200	100	0
4	300	300	200	100	0
5	0	0	0	0	0

row gradient

	1	2	3	4	5
1	-	-	0	0	-
2	-	-	0	0	-
3	90	90	45	26.6	-
4	90	90	63.4	45	-
5	-	-	-	-	-

THETA

	1	2	3	4	5
1	-	-	0	0	-
2	-	-	0	0	-
3	90	90	40	20	-
4	90	90	60	40	-
5	-	-	-	-	-

THETAQ

	1	2	3	4	5
1			3	4	
2			3	4	
3	3	3	4.2	4.8	
4	4	4	4.9	5.6	
5					

D

	1	2	3	4	5
1			3	3	
2			3	3	
3	3	3	3	3	
4	3	3	3	3	
5					

DQ

Accumulator

	0	10	20	30	40	60	...	90
360								
...								
6								
3	4		1		2	1		4
0								

THETAQ

Generalized Hough Transform

- The HT may be extended for any curve with the analytical expression $f(\mathbf{x}, \mathbf{a}) = 0$, where
 - $\mathbf{x}$ is 2D vector with coordinates of a point in the curve
 - $\mathbf{a}$ is a parameter vector
- General algorithm:
 - Initialize the accumulator $A(\mathbf{a})$ with zeros
 - For each pixel $\mathbf{x}$ in the image, compute the values of $\mathbf{a}$, such that $f(\mathbf{x}, \mathbf{a}) = 0$; increment the accumulator: $A(\mathbf{a}) = A(\mathbf{a}) + 1$
 - The local maxima of A corresponds to the curves f in the image